

M37 — Fix TTS audio cut-off on reused primed audio element

Phase: Phase 7 follow-on (post-M36, operator-reported bug fix) **Status:** **DONE** **Blocked by:** M35 (Engage flow that exposed the bug) **Blocks:** none **Estimated effort:** 0.25 day

1. Purpose

Operator-reported bug after M35:

"When engaging a character from the encounter module for a scenario in the multi-patient, the STT input is working but when the selected voice responds, it stops just after it starts the audio response with its reply."

Root cause (client-side, browser audio element state machine).

The TTS client (`portal/static/tts_client.js`) reuses one shared **primed** `<audio>` **element** for every playback — a V6 workaround for Chrome's autoplay policy that would otherwise silently block the async PTT reply (the user gesture from the PTT button press has expired by the time the LLM reply arrives).

The station chat client (`station_chat.js`) plays two utterances back-to-back per turn:

1. A filler ("Hmm.", "One sec.") through the primed element. 2. The full LLM reply through the **same** primed element after `await fillerPromise` resolves.

Setting `audio.src = newUrl` on an `<audio>` element that just finished a playback **does not fully reset its internal state on Chrome/Safari**. Specifically, `ended` stays `true`, `currentTime` stays at the prior duration, and the buffered range from the prior stream lingers. The browser then fires `ended` shortly after the new stream begins (or in some cases immediately on the `loadstart` → `loadedmetadata` transition), and the reply audio cuts off within a fraction of a second of starting.

The bug had been latent in the codebase since V6 introduced the primed-audio workaround. M35's new instructor Engage flow was the first surface that consistently exercised it — operators had previously only hit the chat station via the student `/join` flow, which often had different timing.

2. Structure

Files touched:

- `portal/static/tts_client.js` — `playElevenLabs(text, voiceId,`

```
language) now resets the audio element before assigning the new src: `js const audio
= primedAudio || new Audio(); try { audio.pause(); } catch (e) {} try {
audio.removeAttribute("src"); } catch (e) {} try { audio.load(); } catch (e) {}
try { if (audio.currentTime !== 0) audio.currentTime = 0; } catch (e) {} audio.src
= "/api/tts?" + params.toString(); audio.preload = "auto"; try { audio.load(); }
catch (e) {} currentAudio = audio; ` Six new lines (4 reset steps + 1 post-src load
+ the existing src assignment). Every reset call is wrapped in try/catch because some
browsers throw InvalidStateError when load() is called during certain lifecycle transitions.
```

No new tests other than the JS-source guards. This is a browser-side bug that pytest can't drive directly.

The fix is verified manually by the operator + the M20 Playwright multi- encounter test (when run with `playwright install`) exercises the engage flow end-to-end if extended. Acceptance here is the operator

confirming the reply now plays to completion after the fix lands.

3. Uses

3.1 The full audio-reset sequence

For each call to `playElevenLabs()`:

Step	Call	Purpose
1	<code>audio.pause()</code>	Stop any residual playback. No-op when element is in the just-ended state, defensive.
2	<code>audio.removeAttribute("src")</code>	Detach the prior URL so the next <code>src</code> assignment is treated as a fresh resource (some browsers compare new vs old <code>src</code> and short-circuit if "the same").
3	<code>audio.load()</code>	Force the element's state machine to <code>NETWORK_EMPTY</code> , reset ended, error, etc.
4	<code>audio.currentTime = 0</code>	Belt-and-suspenders — explicit playhead reset in case <code>load()</code> didn't clear it.
5	<code>audio.src = newUrl</code>	Assign the new TTS stream URL.
6	<code>audio.load()</code> (again)	Start the fresh load of the new <code>src</code> .
7	<code>audio.play()</code> (later)	Resume async, returns a Promise.

3.2 Why we keep primedAudio reuse

The V6 design comment in `tts_client.js`:

Chrome (and Safari to a lesser extent) only allows `audio.play()` without a user gesture if the element has been "unlocked" by a prior gesture-driven `play()`. The PTT flow is async (STT → server → LLM → speak), so by the time we try to play the character's reply, the user activation from the PTT button press has expired and `audio.play()` silently fails.

So we cannot switch to new `Audio()` per call without re-introducing the autoplay-block bug. The fix above preserves the primed-element reuse but cleanly resets state between plays.

4. Functions (exported API surface)

No API surface change. The fix is internal to `playElevenLabs`.

Symbol	Where	Purpose
<code>playElevenLabs(text, voiceId, language)</code>	<code>portal/static/tts_client.js</code>	Internal helper called by <code>speak()</code> . M37 adds the reset sequence before the new <code>src</code> assignment.

5. Limitations

- The reset sequence runs even on the **FIRST** playback through

the primed element (where there's nothing to reset). The try/catch wrappers make this safe but it does cost a few extra no-op calls per playback. Imperceptible cost; acceptable.

- `**audio.load()` triggers `loadstart` / `loadedmetadata` / etc.

events on the element.** Our listeners (`playing`, `onended`, `onerror`) don't react to those, so no spurious resolves. If a future module adds a `loadstart` handler to the same element, it will fire spuriously during reset — guard against this when adding such a handler.

- **The fix doesn't address EVERY browser audio bug.** Safari iOS

has additional quirks (e.g. media element can't auto-play HLS in Low Power mode). Acceptable scope — the operator-reported bug was on a desktop browser.

- **Stale event handlers from the previous play.** Each call to

`playElevenLabs` overwrites `audio.onended` and `audio.onerror` with fresh handlers bound to the new Promise's done closure. The previous handlers are gone. The `addEventListener("playing", ..., { once: true })` from the previous play already removed itself when it fired. So stale handlers are not a concern.

6. Test status

Test file	Asserts	Status	Last run		
<code>tests/v7/test_tts_audio_reset.py::test_play_eleven_labs_resets_audio_state_before_new_src</code>	All 4 reset steps appear before the new <code>audio.src = assignment</code>	PASS	2026-05-27		
<code>...::test_play_eleven_labs_calls_load_after_setting_new_src</code>	A second <code>audio.load()</code> appears after the src assignment	PASS	2026-05-27		
<code>...::test_primed_audio_pattern_is_still_used</code>	<code>`primedAudio</code>		new <code>Audio()</code> still in the source — auto play workarounds preserved	PASS	2026-05-27
<code>...::test_reset_calls_are_wrapped_in_try_catch</code>	Each reset call is inside a <code>try { ... } catch block</code>	PASS	2026-05-27		

Test file	Asserts	Status	Last run		
Full v7 suite	243 passed, 1 skipped (M20 Playwright skip — unchanged)	PASS	2026-05-27		

7. Change list

Date	Author	Change	Files
2026-05-27	claude-code	Initial M37 fix: 4-step audio reset sequence + post-src load() before each playElevenLabs playback; 4 source-guard tests	portal/static/tts_client.js, tests/v7/test_tts_audio_reset.py (new)

8. Open questions / known issues

- **Operator verification required.** Pytest can't drive a real

browser to confirm the cut-off no longer occurs. Operator should exercise the Engage flow in multi-patient mode and confirm the reply audio plays to completion. If cut-off persists, the next diagnostic step is to instrument `tts_client.js` to log `audio.duration`, `audio.currentTime`, and the `ended/error` event order to the console during reply playback.

- **Potential SECOND cause (not addressed here):** the

`_session_el_key()` helper in `portal/server.py` calls `control_session.get_active()`, which (per M2) returns `None` in multi-encounter rooms. This means `/api/tts` falls through to `voices.get_api_key(None)` which uses `env / keyfile / runtime cache` — NOT the per-encounter `elevenlabs_api_key` set at room start time. If the operator stored the key only in the vault (no env fallback) AND has never visited an operator route during this server lifetime, the runtime cache could be empty and TTS would 503-fallback to browser voice. We did NOT change this in M37 because the operator reported "the selected voice responds" — implying ElevenLabs IS being reached. But if the key-resolution path turns out to also play a role, a future M38 should plumb the encounter's `elevenlabs_api_key` through `/api/tts` via the station's `join_code` (which is already in the URL).

- **Filler latency vs. fix overhead.** The reset sequence adds a

small amount of work between filler end and reply start (~milliseconds). Imperceptible in practice; the user's perception of the reply start is dominated by the LLM round-trip + first-chunk latency (200+ ms). Document the budget here so future audio refactors can compare.

Render this guide to PDF with `python docs/module_guides/render_pdfs.py`.